FAZENDA AURORA · APOSTILA DO ALUNO

UC 01

C++: a lógica da fazenda

Transforme problemas da fazenda em algoritmos, decisões e programas claros.

Aprender para transformar

Você chegou a uma fazenda que precisa de organização, decisões e automação. Cada capítulo transforma uma necessidade da Aurora em uma habilidade prática. Leia, altere os exemplos, teste e explique o seu raciocínio.

Como usar esta apostila

1. Leia a situação e os conceitos. 2. Reproduza o exemplo no ambiente indicado. 3. Faça a missão sem apenas copiar a resposta. 4. Teste situações comuns e limites. 5. Use o checkpoint para revisar e a mentora para discutir dúvidas.

Nesta UC

01 Primeiro sinal de vida

02 Variáveis: o inventário do celeiro

03 Entradas e contas: a água necessária

04 Decisões: irrigar ou esperar?

05 Repetições: visitar todos os canteiros

06 Funções: uma regra que podemos reutilizar

07 Vetores: o mapa das leituras

08 Missão final: a central de irrigação

Os exemplos são didáticos. A avaliação da IA é formativa e deve ser confrontada com testes. Os laboratórios do navegador não substituem um compilador C++, uma VM ou a montagem no Tinkercad.

CAPÍTULO 01 · 15 MIN DE LEITURA E PRÁTICA

Primeiro sinal de vida

O painel da Fazenda Aurora está apagado. Antes de automatizar qualquer coisa, precisamos saber que o programa está funcionando.

Entenda o conceito

Um algoritmo é uma sequência de passos para resolver um problema. Pense em entrada, processamento e saída: receber o nome de um setor, preparar uma mensagem e exibi-la.

C++ é uma linguagem compilada. Você escreve um arquivo .cpp; um compilador como g++ verifica a sintaxe e produz um programa executável. O navegador deste curso não compila C++: use a máquina virtual da UC 2 para executar seus programas.

#include <iostream> disponibiliza entrada e saída. A função main é o ponto inicial. As chaves delimitam blocos; muitas instruções terminam em ponto e vírgula. std::cout envia dados para a saída padrão.

Exemplo para experimentar

```
#include <iostream>

int main() {
    std::cout << "Aurora online!\n";
    return 0;
}
```

Sua missão na Aurora

Crie um programa que exiba Aurora online! e, em outra linha, o nome de um setor da fazenda. Escreva também o que main e cout fazem.

Antes de enviar

- Estrutura main válida
- Duas saídas em linhas distintas
- Explicacão de main e cout

Atenção: Não coloque o texto fora das aspas e não esqueça o ; ao final de cout.

Checkpoint

Qual função marca o ponto de entrada deste programa?

A) start()

B) main()

C) cout()

CAPÍTULO 02 · 15 MIN DE LEITURA E PRÁTICA

Variáveis: o inventário do celeiro

Lia, a responsável pela fazenda, precisa saber quantos sacos de ração e litros de água estão disponíveis.

Entenda o conceito

Uma variável associa um nome a um valor. Declare seu tipo e inicialize antes de usar. int representa números inteiros; double representa números com parte fracionária; bool representa verdadeiro ou falso.

std::string armazena texto e exige <string>. Use nomes que expliquem o significado: litrosAgua é mais claro que x. O tipo faz parte do raciocínio: contar sacos e medir litros são problemas diferentes.

O operador = atribui; não faz uma comparação. Ao atualizar uma quantidade, pense na unidade de medida e no estado anterior. Evite misturar litros com mililitros sem converter.

Exemplo para experimentar

```
#include <iostream>
#include <string>

int main() {
    int sacos = 12;
    double litrosAgua = 85.5;
    bool portaoAberto = false;
    std::string setor = "Celeiro";
    std::cout << setor << ": " << sacos << " sacos\n";
    std::cout << litrosAgua << " litros\n";
    return 0;
}
```

Sua missão na Aurora

Declare o nome de um setor, 24 sacos de ração, 37,5 litros de água e o estado fechado de um portão. Exiba as informações com as unidades corretas.

Antes de enviar

- Tipos adequados
- Inicialização dos quatro valores
- Saída com unidades

Atenção: No código, a parte decimal usa ponto, não vírgula. Uma variável sem inicialização pode ter um valor indeterminado.

Checkpoint

Qual tipo combina com 37,5 litros de água?

A) int

B) double

C) bool

CAPÍTULO 03 · 15 MIN DE LEITURA E PRÁTICA

Entradas e contas: a água necessária

O tamanho das plantações varia. Seu programa deve calcular a necessidade de água a partir de uma entrada.

Entenda o conceito

std::cin lê valores da entrada padrão. Para um número, cin >> quantidade tenta interpretar o que foi digitado. Antes de usar esse valor em uma decisão importante, confirme que a leitura deu certo.

Multiplicação usa *, divisão usa / e resto de divisão inteira usa %. A expressão 5 / 2, com inteiros, vale 2. Já 5.0 / 2 vale 2.5. Escolher o tipo correto evita perder a parte fracionária.

Um bom teste tem casos comuns e casos de borda. Para consumo, teste zero, um número positivo e uma entrada inválida. Se o enunciado proíbe números negativos, seu programa precisa rejeitá-los.

Exemplo para experimentar

```
#include <iostream>

int main() {
    double area = 0;
    std::cout << "Area em m2: ";
    if (!(std::cin >> area) || area < 0) {
        std::cout << "Entrada invalida\n";
        return 1;
    }
    double litros = area * 2.5;
    std::cout << "Necessidade: " << litros << " litros\n";
    return 0;
}
```

Sua missão na Aurora

Leia a área em m² e calcule 2,5 litros por m². Rejeite áreas negativas e leituras inválidas. Mostre testes para 0, 10 e -1.

Antes de enviar

- Leitura validada
- Cálculo e unidade corretos
- Três testes explicados

Atenção: Não use um valor depois de uma leitura que falhou. Também não confunda % com porcentagem.

Checkpoint

Com operandos inteiros, quanto vale 5 / 2 em C++?

A) 2

B) 2.5

C) 3

CAPÍTULO 04 · 15 MIN DE LEITURA E PRÁTICA

Decisões: irrigar ou esperar?

A plantação não pode receber água o tempo todo. É hora de transformar uma regra em uma decisão.

Entenda o conceito

if executa um bloco quando sua condição é verdadeira. else oferece o caminho alternativo. Use <, <=, >, >=, == e != para comparar.

As fronteiras importam: umidade < 40 não inclui 40. O operador == compara; = atribui. Escreva a condição antes do código usando uma frase precisa.

Uma decisão automática deve ter uma justificativa visível. Exibir o estado e a razão facilita testar e localizar problemas. Neste treino, a regra é irrigar abaixo de 40% de umidade.

Exemplo para experimentar

```
#include <iostream>

int main() {
    int umidade = 0;
    if (!(std::cin >> umidade) || umidade < 0 || umidade > 100) {
        std::cout << "Umidade invalida\n";
        return 1;
    }
    if (umidade < 40) {
        std::cout << "IRRIGAR\n";
    } else {
        std::cout << "AGUARDAR\n";
    }
    return 0;
}
```

Sua missão na Aurora

Leia a umidade entre 0 e 100. Abaixo de 40, mostre IRRIGAR; nos demais valores válidos, AGUARDAR. Explique as saídas para 39, 40 e 41.

Antes de enviar

- Condição estrita < 40
- Validação 0 a 100
- Testes 39, 40 e 41

Atenção: Uma única igualdade errada muda a regra. Teste exatamente o limite, não apenas um valor bem distante dele.

Checkpoint

Com a regra umidade < 40, o que acontece em 40?

A) Irriga

B) Aguarda

C) Sempre dá erro

CAPÍTULO 05 · 15 MIN DE LEITURA E PRÁTICA

Repetições: visitar todos os canteiros

Há vários canteiros para inspecionar. Reescrever a mesma instrução não é uma boa solução.

Entenda o conceito

for combina inicialização, condição e atualização. É útil quando você conhece a quantidade de repetições. while repete enquanto uma condição for verdadeira.

Um laço precisa avançar em direção ao término. Esquecer a atualização pode gerar um laço infinito. Os índices de um conjunto de tamanho n normalmente vão de 0 a n - 1.

Um acumulador reúne valores sucessivos. Inicie a soma em zero; em cada passagem, adicione a leitura atual. Para calcular uma média, divida pela quantidade, tratando o caso de quantidade zero.

Exemplo para experimentar

```
#include <iostream>

int main() {
    int soma = 0;
    for (int canteiro = 1; canteiro <= 3; ++canteiro) {
        int umidade = 0;
        if (!(std::cin >> umidade) || umidade < 0 || umidade > 100) return 1;
        soma += umidade;
    }
    double media = soma / 3.0;
    std::cout << "Media: " << media << "%\n";
    return 0;
}
```

Sua missão na Aurora

Leia cinco umidades válidas, calcule a média sem perder os decimais e conte quantas estão abaixo de 40. Use um laço, não cinco blocos copiados.

Antes de enviar

- Laço com cinco leituras
- Média com decimais
- Contagem dos canteiros secos

Atenção: Confira quantas vezes o laço roda. <= e < podem produzir uma repetição a mais ou a menos.

Checkpoint

Por que dividir a soma por 5.0, e não por 5, quando a soma é int?

A) Para fazer divisão com parte fracionária

B) Para somar mais rápido

C) Para impedir toda entrada inválida

CAPÍTULO 06 · 15 MIN DE LEITURA E PRÁTICA

Funções: uma regra que podemos reutilizar

A regra de irrigação precisa funcionar em vários setores sem ser copiada em todo lugar.

Entenda o conceito

Uma função recebe parâmetros, executa uma tarefa e pode devolver um resultado. Separe calcular de exibir: isso facilita testar a lógica com entradas conhecidas.

&& significa E: as duas condições devem ser verdadeiras. || significa OU: basta uma condição verdadeira. ! inverte um valor lógico. Use parênteses para tornar condições compostas claras.

A função deve ter uma responsabilidade definida. Uma regra deve consultar a umidade e o nível do reservatório e retornar bool; quem chama a função decide como apresentar o resultado.

Exemplo para experimentar

```
#include <iostream>

bool deveIrrigar(int umidade, int nivel) {
    return umidade < 40 && nivel >= 20;
}

int main() {
    std::cout << deveIrrigar(30, 70) << "\n";
    std::cout << deveIrrigar(30, 10) << "\n";
    return 0;
}
```

Sua missão na Aurora

Escreva deveIrrigar(umidade, nivel). Ela deve retornar verdadeiro apenas quando a umidade for menor que 40 e o reservatório tiver pelo menos 20%. Teste quatro combinações.

Antes de enviar

• Função retorna bool

• Conjunção dos dois requisitos

• Teste seco/com água, seco/sem água, úmido/com água e limite

Atenção: Uma condição com || poderia ligar a irrigação mesmo com o reservatório vazio.

Checkpoint

Qual condição representa os dois requisitos ao mesmo tempo?

A) umidade < 40 || nivel >= 20

B) umidade < 40 && nivel >= 20

C) umidade = 40

CAPÍTULO 07 · 15 MIN DE LEITURA E PRÁTICA

Vetores: o mapa das leituras

A central precisa guardar as leituras dos canteiros, não apenas exibir cada uma e esquecer.

Entenda o conceito

std::vector é uma coleção de tamanho variável. Inclua <vector>. Você pode inicializar com valores, percorrer com for e usar size() para descobrir a quantidade.

O primeiro índice é zero. Para três leituras, os índices válidos são 0, 1 e 2. Acessar um índice inexistente com [] causa comportamento indefinido; at() verifica o limite e pode lançar uma exceção.

Um for baseado em intervalo é uma forma clara de ler todos os valores sem gerenciar um índice. Ao calcular estatísticas, valide os valores e cuide do vetor vazio.

Exemplo para experimentar

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> leituras = {35, 61, 27, 80};
    int secos = 0;
    for (int umidade : leituras) {
        if (umidade < 40) ++secos;
    }
    std::cout << secos << " canteiros secos\n";
    return 0;
}
```

Sua missão na Aurora

Guarde as leituras 35, 61, 27 e 80 em um vector. Mostre o total de leituras, a média e a quantidade de canteiros secos. Explique qual é o último índice válido.

Antes de enviar

- Uso de vector
- Média e contagem corretas
- Índices e limites explicados

Atenção: Não percorra com i <= tamanho: o índice igual ao tamanho já está fora da coleção.

Checkpoint

Qual é o último índice de um vector com quatro elementos?

A) 4

B) 3

C) 1

CAPÍTULO 08 · 25 MIN DE LEITURA E PRÁTICA

Missão final: a central de irrigação

A lógica da Aurora agora pode virar uma central com leituras, decisões e um relatório diário.

Entenda o conceito

Integre as etapas: leia entradas, valide, calcule decisões e exiba o relatório. Organize a regra em uma função e as leituras em um vector.

Uma solução testável define comportamento para valores de borda e para entradas inválidas. Não confunda programa que compilou com programa que resolveu o problema.

Documente como executar, quais unidades são usadas e quais testes foram feitos. Legibilidade e explicação do raciocínio contam na avaliação, além da saída correta.

Exemplo para experimentar

```
#include <iostream>
#include <vector>

bool deveIrrigar(int umidade, int nivel) {
    return umidade < 40 && nivel >= 20;
}

int main() {
    int nivel = 0;
    if (!(std::cin >> nivel) || nivel < 0 || nivel > 100) return 1;
    std::vector<int> leituras(4);
    for (int& leitura : leituras) {
        if (!(std::cin >> leitura) || leitura < 0 || leitura > 100) return 1;
    }
    for (int i = 0; i < 4; ++i) {
        std::cout << "Canteiro " << i + 1 << ": "
                  << (deveIrrigar(leituras[i], nivel) ? "IRRIGAR" : "AGUARDAR")
                  << "\n";
    }
    return 0;
}
```

Sua missão na Aurora

Entregue uma central para quatro canteiros: leia o reservatório e quatro umidades, valide todos os percentuais, use uma função e mostre um relatório por canteiro. Inclua testes para nível 19, nível 20, umidade 39 e umidade 40.

Antes de enviar

- Entradas válidas e inválidas tratadas
- Função e vector usados com clareza
- Relatório por canteiro
- Casos de borda demonstrados

Atenção: Um relatório bonito não substitui os testes. Não altere o limiar para esconder um teste que falhou.

Checkpoint

Compilar sem erros garante que a regra está correta?

A) Sim

B) Não; precisamos testar o comportamento

C) Só se houver comentários

Avaliação e testes

A nota formativa vai de 0 a 100. Não é um certificado nem uma avaliação oficial da escola. O mais importante é usar o retorno para revisar a solução e demonstrar que ela funciona.

Solução do problema: até 40 pontos

Lógica e domínio dos conceitos: até 25 pontos

Testes e casos de borda: até 20 pontos

Clareza e explicação: até 15 pontos

Modelo de tabela de testes

Para cada teste, registre: entrada ou situação; resultado esperado; resultado observado; conclusão. Um caso que falhou é uma informação útil para melhorar, não algo a esconder.

Como conversar com a mentora

Diga em qual missão está, cole um trecho pequeno e informe o erro ou a saída observada. Peça uma pista antes de pedir a solução completa. Não envie senhas, tokens ou dados pessoais de outras pessoas.

Gabarito comentado dos checkpoints

01. B — A execução começa em main. cout é um objeto usado para escrever na saída.

02. B — double preserva uma parte fracionária. No código C++, use 37.5.

03. A — A divisão entre inteiros descarta a parte fracionária; não arredonda para o inteiro mais próximo.

04. B — 40 não é menor que 40; portanto segue o caminho de espera.

05. A — O operando 5.0 faz a operação considerar ponto flutuante.

06. B — && exige que as duas condições sejam satisfeitas.

07. B — Como a contagem começa em zero, o último índice é tamanho - 1.

08. B — O compilador verifica muitas regras da linguagem, mas não sabe qual era o problema que você deveria resolver.

Referências e ferramentas oficiais

GNU GCC · compilador C++

Ubuntu · VM com VirtualBox

VirtualBox · download oficial

Ubuntu 24.04 LTS · imagens oficiais

Arduino · referência da linguagem

Tinkercad Circuits